PSAI_121

ink! Analyzer

- **Team Name:** David Semakula
- **Payment Address:** 0x5c1D926f62B61C9cf62167474111385ff7393c61 (USDC)
- **Level:** 2

Project Overview :page_facing_up:

Overview

ink! analyzer is a collection of modular and reusable libraries and tools for semantic analysis of ink! smart contract code.

Objective

ink! analyzer aims to improve ink! language support in integrated development environments (IDEs), source code editors and other development tools by providing modular and reusable building blocks for implementing features like diagnostic errors, code completion suggestions, code/intent actions and hover content for the ink! programming language which is used for writing smart contracts for blockchains built on Substrate.

Problem

ink! is an Embedded Domain Specific Language (eDSL) that you can use to write WebAssembly based smart contracts in the Rust programming language. In fact, "ink! is just standard Rust in a well-defined"contract format" with specialized `#[ink(…)]` attribute macros".

This allows ink! developers to leverage Rust tooling like clippy, cargo, crates.io and excellent IDE/code editor support via rust-analyzer and IntelliJ Rust.

However, relying on only generic Rust language support in IDEs, code editors and other development tools has some significant limitations for the developer experience including:

- No language support (e.g. diagnostic errors) for ink!'s domain specific semantic rules for smart contracts (e.g. exactly one `#[ink(storage)]` struct, at least one `#[ink(message)]` method and the same for `#[ink(constructor)]`, ink! attributes should be applied to items of the correct type e.t.c)
- Inconsistent editor experience with issues like no code completion and/or hover content for some ink! attribute macro arguments (e.g `#[ink(payable)]`) because macro expansion/name resolution and trait resolution are hard problems for generic IDE/code editor tools (see also https://rust-analyzer.github.io/blog/2021/11/21/ides-and-macros.html).

Solution

To solve the above challenges and improve ink! language support in IDEs, code editors and other development tools, this project will create two main components: - a modular domain-specific semantic analysis library for ink!. - a Language Server Protocol (LSP) implementation built on top of the aforementioned semantic analysis library.

These two components can then be reused to add ink! language support to multiple IDEs, code editors and other development tools.

In particular, a large number of IDEs and code editors support LSP servers, these include Visual Studio Code, Visual Studio, Vim / Neovim, Emacs, Atom, Sublime Text, Acme, Lapce, Eclipse and many more. This project makes it relatively easy for any developer to write extensions/plugins/integrations that add ink! language support to any tool with support for LSP servers.

While for IDEs, code editors and other development tools that don't support LSP servers, extensions/plugins/integrations can either implement their own LSP client middleware or use the semantic analyzer directly. However, this grant application will focus on an LSP based architecture.

Lastly, this project will implement a Visual Studio Code extension that adds ink! language support to the editor as a reference implementation and showcase. However, other developers will be encouraged to leverage the above components to add ink! language support to their preferred IDE, code editor or other development tool.

Observation

In the context of smart contracts and the blockchain ecosystem, static code analysis tends to be closely associated with security and vulnerability detection. This project uses the narrower term "semantic analysis" because its primary goal (at this stage) is to improve ink! language support in IDEs, code editors and other development tools. Therefore, initially, ink! analyzer will not provide any features related to security and vulnerability detection. However, such functionality may be added in the future.

Project Details

Architecture Diagram



Figure 1: ink! Analyzer - LSP based Architecture Diagram

Architecture Components

1. Semantic Analyzer

The semantic analyzer is responsible for parsing the smart contract code, analyzing it based on ink!'s semantic rules for smart contracts and returning semantic information for other components to consume.

It will be written in Rust and will use rust-analyzer's ra_ap_syntax crate for generating the syntax tree of the smart contract code that it analyzes. It will have an internal architecture similar to rust-analyzer's ra_ap_ide crate with notions of persistent state, change over time, immutable snapshots of state and cancellable transactions.

While ink!'s ink_ir crate includes syntax tree and intermediate representation (IR) modules, the semantic analyzer will use rust-analyzer libraries because the ink_ir crate uses syn as its parser and syn (being "geared toward use in Rust procedural macros") assumes "that all input to it is well-formed". This makes it unsuitable for our IDE/code editor use case, because among other design goals for this use case, parsing should be: - resilient (even if the input is invalid, parser tries to see as much syntax tree fragments in the input as it can). - lossless (even if the input is invalid, the tree produced by the parser represents it exactly).

Because of these and other design goals, rust-analyzer created its own parsing and syntax tree libraries instead of using the rust compiler's original libsyntax parser or syn as articulated in the RFC that initiated its development. ink! analyzer follows the same rationale in choosing rust-analyzer's libraries over ink_ir's syn based modules.

As a concrete example, for the !ink/Rust code snippet below that contains only one invalid line, syn simply returns an error while ra_ap_syntax still generates a lossless syntax tree with errors represented by "error nodes", thus allowing analysis to still be performed on the rest of the valid code.

```
#[ink::contract]
mod flipper {

    #[ink(storage)]
    pub struct Flipper {
        value: bool,
    }

    bad statement; // This is an invalid line
}
```

However, ink_ir and related crates and modules will still be used as reference implementations for ink! semantic rule validation.

2. Language Server

The language server implements the Language Server Protocol (LSP) and acts as a backend that provides language support features like diagnostic errors, code completion suggestions, code/intent actions and hover content to IDEs, code editors and other development tools. It uses the semantic analyzer as the engine for providing ink! language support features by: - translating LSP requests into semantic analyzer interface calls. - translating semantic analysis results into corresponding LSP types.

It can be reused by multiple IDEs, code editors and other development tools that support LSP servers including Visual Studio Code, Visual Studio, Vim / Neovim, Emacs, Atom, Sublime Text, Acme, Lapce, Eclipse and many more.

It will be written in Rust and will use rust-analyzer's lsp-server crate to handle LSP protocol handshaking and parsing messages.

3. Extension/Plugin/Integrations for IDE, Code Editor or Development Tool

Extensions/plugins/integrations add ink! language support to a specific IDE, code editor or development tool by communicating with the language server running as a separate process using the Language Server Protocol via JSON-RPC.

Only a VS Code extension will be implemented at this stage of the project. The VS Code extension will be written in TypeScript using VS Code's extension API.

Technology Stack

Rust - will be used for implementing the semantic analyzer and language server components.

TypeScript - will be used for implementing the VS Code extension.

Ecosystem Fit

This project will provide building blocks for improving ink! language support in IDEs, code editors and other development tools thus improving the developer experience for ink! smart contract developers. It will additionally implement an extension for VS Code, one of the most popular code editors for developers, that will also serve as a showcase and reference implementation that other developers can use as a guide for adding ink! language support to their preferred IDE or code editor if it supports LSP servers.

I'm not currently aware of any similar projects in the Substrate / Polkadot / Kusama ecosystem. However, for Solidity and the Ethereum ecosystem, there are many IDE and code editor extensions/plugins (e.g. this, this and this for Visual Studio Code and Solidity plugin for IntelliJ), a few language server implementations (e.g. this and this which are both used by their respective VS Code extensions) and a few static code analyzers, including Remix Analyzer which is specifically built for IDEs and code editors.

NOTE: The only open source smart contract static code analyzer for the Substrate / Polkadot / Kusama ecosystem I could find is Vanguard, however it operates on LLVM bytecode and is therefore not usable as a building block for improving high-level language support in IDEs, code editors and other development tools.

Team :busts_in_silhouette:

Team members

- David Semakula davidsemakula

Contact

- **Contact Name:** David Semakula
- **Contact Email:** hello@davidsemakula.com
- **Website:** https://davidsemakula.com

Legal Structure

I'll be working as an individual.

- **Registered Address:** N/A
- **Registered Legal Entity:** N/A

Team's experience

I'm a Software Engineer & Systems Architect with over 10 years of experience. I've worked as a technical lead on projects for HubSpot, Permobil, Pressboard, Grindery, InboundLabs, Tunga, ButterflyWorks, Telegraaf Media Groep (TMG) and many more companies. I've also won (or been part of teams that have won) prizes at a few hackathons including a category top prize at the recent Moonbeam Illuminate/22 Hackathon. I hold a Bachelor of Science in Computer Science degree from Makerere University, Kampala, Uganda. You can find my full profile at https://davidsemakula.com.

Team Code Repos

- https://github.com/davidsemakula
- https://github.com/ink-analyzer/ink-analyzer
- https://github.com/davidsemakula/substrate-vault
- https://github.com/mukutu-tech/mukutu-safe

Team LinkedIn Profiles (if available)

- https://davidsemakula.com (I use my own website for my profile)

Development Status :open_book:

I've developed a proof of concept (POC) for the semantic analyzer that can be accessed on Github at https://github.com/ink-analyzer/ink-analyzer. The POC implements a diagnostic that detects when the `#[ink::contract]` attribute is applied to anything other than a `mod` item and returns a diagnostic model that includes an error message and the text range to which the diagnostic applies.

Development Roadmap :nut_and_bolt:

Overview

- **Total Estimated Duration:** ~4.25 months
- **Full-Time Equivalent (FTE):** 1
- **Total Costs:** 30,000 USDC

Milestone 1 — Semantic Analyzer: Setup and Diagnostics

- **Estimated duration:** ~3 weeks
- **FTE:** 1
- **Costs:** 5,600 USDC

Number	Deliverable	Specification
0a.	License	Apache 2.0 and MIT
0b.	Documentation	I will provide detailed source documentation including rustdoc documentation that will be published to docs.rs and a README file (published on on Github, crates.io and docs.rs) providing general information about the library, instructions for installing and using the library and links to other documentation and resources related to the library.
0c.	Testing and Testing Guide	Core functions will be fully covered by comprehensive unit tests to ensure functionality and robustness. In the guide, I will describe how to run these tests.
0d.	Docker	I will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
1.	Rust library crate: Diagnostic errors	I will create a Rust library crate with an interface that accepts ink! smart contract code as input and returns diagnostic errors based on ink!'s semantic rules. Diagnostic errors will include: conflicting ink! attributes and/or arguments based on the item being annotated and list of ink! attribute arguments (e.g `#[ink(storage)]` applied to anything other than a struct), missing required ink! attributes (e.g no `#[ink(constructor)]`, `#[ink(storage)]` or `#[ink(message)]` in a contract mod), multiple annotations of attributes and/or arguments that can only be applied to one item in a contract (e.g multiple `#[ink(storage)]` structs or more than one wildcard selector among `#[ink(message)]`, as well as `#[ink(constructor)]` methods), duplicate ink! attributes on an item (e.g multiple `#[ink(message)]` annotations on one method), overlapping `#[ink(constructor)]`and `#[ink(message)]` selectors.

Milestone 2 — Semantic Analyzer: Code completion suggestions, code/intent actions and hover content

- **Estimated duration:** ~5-6 weeks
- **FTE:** 1
- **Costs:** 10,800 USDC

Number	Deliverable	Specification
0a.	License	Apache 2.0 and MIT
0b.	Documentation	I will provide detailed source documentation including rustdoc documentation that will be published to docs.rs and a README file (published on on Github, crates.io and docs.rs) providing general information about the library, instructions for installing and using the library and links to other documentation and resources related to the library.
0c.	Testing and Testing Guide	Core functions will be fully covered by comprehensive unit tests to ensure functionality and robustness. In the guide, I will describe how to run these tests.
0d.	Docker	I will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
1.	Rust crate update: Code completion suggestions	I will update the semantic analyzer crate to provide an interface that accepts ink! smart contract code and a cursor position as input and returns code completion suggestions for relevant ink! attributes and arguments.
2.	Rust crate update: Code/intent actions	I will update the semantic analyzer crate to provide an interface that accepts ink! smart contract code and a cursor position as input and returns code/intent actions for adding relevant ink! attributes and arguments.
3.	Rust crate update: Hover content	I will update the semantic analyzer crate to provide an interface that accepts ink! smart contract code and a text range as input and returns documentation for relevant ink! attributes and arguments.

Milestone 3 — Language Server

- **Estimated duration:** ~3-4 weeks
- **FTE:** 1
- **Costs:** 6,800 USDC

Number	Deliverable	Specification
0a.	License	Apache 2.0 and MIT
0b.	Documentation	I will provide detailed source documentation including rustdoc documentation that will be published to docs.rs and a README file (published on on Github, crates.io and docs.rs) providing general information about the crate, instructions for installing and using the crate and links to other documentation and resources related to the crate.
0c.	Testing and Testing Guide	Core functions will be fully covered by comprehensive unit tests to ensure functionality and robustness. In the guide, I will describe how to run these tests.
0d.	Docker	I will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
1.	Rust binary crate: ink! Language Server Protocol implementation	I will create Rust binary crate that implements the Language Server Protocol for the ink! language support features provided by the semantic analyzer as outlined in milestones 1 and 2 above (i.e diagnostic errors, code completion suggestions, code/intent actions and hover content as defined in the respective milestones above).

Milestone 4 — Visual Studio Code Extension

- **Estimated duration:** ~3-4 weeks
- **FTE:** 1
- **Costs:** 6,800 USDC

Number	Deliverable	Specification
0a.	License	GPL-3.0
0b.	Documentation	I will provide inline source documentation and a README file (published on on Github and the VS Code marketplace) providing general information about the extension, instructions for installing and using the extension and links to other documentation and resources related to the extension.
0c.	Testing and Testing Guide	Core functions will be fully covered by comprehensive integration tests to ensure functionality and robustness. In the guide, I will describe how to run these tests.
0d.	Docker	I will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
0e.	Article	I will publish an **article** that explains how to use all the components of ink! analyzer, including the VS Code extension, the language server and the semantic analyzer.
1.	Visual Studio Code Extension	I will create a Visual Studio Code Extension that supports the ink! language support features provided by the language server and semantic analyzer as outlined in milestones 1, 2 and 3 above (i.e diagnostic errors, code completion suggestions, code/intent actions and hover content as defined in the respective milestones above).

Future Plans

I will publish the VS Code extension to the VS Code marketplace and share it along with the code and documentation for all ink! analyzer components in Substrate / Polkadot / Kusama ecosystem developer communities.

In the short-term, I will likely apply for follow-up funding for: - adding more features to existing components. - maintaining existing components. - adding new reusable components for tools that don't support LSP.

As the project is a public good, the long-term goal is to create a community of users, contributors and ecosystem partners who are invested in the project's long-term success because of its utility.

Referral Program (optional) :moneybag:

N/A

Additional Information :heavy_plus_sign:

How did you hear about the Grants Program?

Web3 Foundation Website.